

VRTX Ledger System – Assignment

Objective

Build a ledger-based financial system using Java (Spring Boot) and PostgreSQL. The system should reflect real-world fintech constraints such as consistency, traceability, and correctness.

API Style

You are expected to build RESTful APIs using Spring Boot (JSON over HTTP).

Key Accounting Concepts

Double-entry accounting means every transaction affects at least two accounts: one debit and one credit, and totals must always match.

- 1 Debit: Money leaving an account
- 2 Credit: Money entering an account
- 3 Sum of debits must equal sum of credits
- 4 Ledger is append-only (no update/delete)

Example: Card Payment

User pays 100. Merchant receives 97. Fees = 3.

- 1 Debit: User account → 100
- 2 Credit: Merchant account → 97
- 3 Credit: Fees account → 3

Balanced transaction: $100 = 100$

Core Requirements

- 1 Accounts (User, Merchant, Fees, Settlement)
- 2 Double-entry ledger
- 3 Immutable transactions

Supported Scenarios

- 1 Card payment (POS / contactless)
- 2 Refund / reversal
- 3 Wallet transfer
- 4 Merchant settlement
- 5 Fees handling

Technical Constraints

- 1 Balanced transactions
- 2 Idempotency with idempotency_key

- 3 Concurrency handling
- 4 Atomic operations

Advanced Requirements

- 1 Transaction linking
- 2 Ledger-based balance calculation
- 3 Concurrency safety
- 4 Reconciliation mechanism

Required APIs

- 1 POST /transactions/payment
- 2 POST /transactions/refund
- 3 POST /transactions/transfer
- 4 POST /transactions/settlement
- 5 GET /accounts/{id}/ledger

Deliverables

- 1 PostgreSQL schema
- 2 Spring Boot REST APIs
- 3 Transaction logic
- 4 Edge case handling

Submission Instructions

Create a private GitHub repository and invite the following users:

- 1 @omarvrtx
- 2 @Abdel-Monaam-Aouini

Bonus (Optional)

- 1 Multi-currency
- 2 Configurable fees
- 3 Event-driven design